

Petri Net Scheduling of FMS Using Branch and Bound Method*

MengChu Zhou, Hua-Sheng Chiu and H. Henry Xiong

Department of Electrical and Computer Engineering
Center for Manufacturing Systems
New Jersey Institute of Technology
Newark, NJ 07102-1982, USA
zhou@njit.edu

Abstract - In this paper, Petri net models for Flexible Manufacturing Systems (FMS) are constructed. A firing sequence of the Petri net from an initial marking to final one can be seen as a schedule of the modeled FMS. By using the branch-and-bound algorithm, an optimal schedule of the FMS can be obtained. Timed marked graphs, a special class of timed Petri nets, are then used to analyze the long-term performance of the job-shop systems. For job shop system, a procedure is proposed to derive the marked graph from the initial Petri net model after the schedule is determined. The branch and bound algorithm and proposed procedure are illustrated through a job-shop system.

I. INTRODUCTION

Production scheduling concerns the efficient allocation of resources over time for the manufacturing of goods. Scheduling problems arise whenever a common set of resources— material, machine, and robot — must be used to make a variety of products. The objective of scheduling is to find a way to assign and sequence the use of these shared resources such that production constraints are satisfied and the time spent is minimized. Better production schedules provide a competitive advantage through gains in resource productivity and related efficiencies in operations management. Competition has also motivated the introduction of sophisticated and capital intensive new manufacturing systems made possible by the declining cost and increasing power of industrial computers and robots.

The formal job-shop or machine-scheduling problem may be stated as follows: N jobs are to be processed on M machines. Each job consists of a set of operations, one operation uniquely associated with one of the M machines. The processing time for an operation cannot be split. Technological constraints demand that the operations within each job must be processed in a unique order. The scheduling problem involves determining the sequence and timing of each operation on each machine such that some given performance criterion is maximized or minimized. Typical performance goals include minimizing the

makespan, i.e., minimizing the time required to complete all of the jobs [14].

This paper emphasizes the Petri net approach for modeling, control, and performance analysis of flexible manufacturing systems (FMS). This approach has become more important in recent years because it can solve problems that cannot be modeled using queuing theory, and avoid the time consuming, trial and error approach of simulation.

The mathematical model of an FMS cell has been synthesized using Petri nets. The concurrency, conflicts, resource-sharing, and sequential operations have been built in the net models. The resulting net models have the system properties such as liveness, safeness, and reversibility. Application of Petri net theory to practical FMS is a very important area in order to develop modern manufacturing control systems [19].

The purpose of this paper is to demonstrate how Petri nets can be used in design of flexible manufacturing systems. Five objectives are as follows:

- 1) To present the branch-and-bound algorithm to solve the scheduling problem.
- 2) To discuss modeling, analysis, and control issues of FMS with Petri nets and summarize the Petri net design approaches proposed in the literature.
- 3) To analyze the system cycle time based on the resulting Petri net model.

II. PETRI NET MODELING

Mathematical modeling or synthesis of FMS is the first step to analyze, simulate, and control the operations of such systems. When concurrency and synchronization concepts become keys for study of modern manufacturing system control, Petri nets provide a most straightforward tool to model these concepts. In addition, they can model conflicts, nondeterminism, timing information, and resource-sharing environments.

Petri nets are a graphical tool where places, pictured as circles, are used to represent availability of resources, operation processes, or conditions, and transitions, pictured as bars, model the events, start, or termination of operations.

* Partially supported by NSF Grant DMI-9410386 and the Center for Manufacturing Systems at NJIT.

Arcs indicated the relationship between places and transitions. Tokens in places and their flow regulated by firing transitions add the dynamics to Petri nets. Thus a marked Petri net can be used to study dynamic behavior of the modeled discrete event systems [19].

For a fundamental knowledge of Petri net theory, a reader is referred to [12]. Strictly, a Petri net may be defined as $Z = (P, T, I, O, m)$, where

- (1) $P = \{p_1, p_2, \dots, p_n\}$, $n > 0$;
- (2) $T = \{t_1, t_2, \dots, t_s\}$, $s > 0$;
- (3) $I : P \times T \rightarrow \{0, 1\}$;
- (4) $O : P \times T \rightarrow \{0, 1\}$;
- (5) $m : P \rightarrow \{0, 1, 2, \dots\}$;

In this definition, p_i is called a place, t_i a transition, I an input function defining the set of directed arcs from P to T , O an output function defining the set of directed arcs from T to P , and m is an n -dimensional marking whose i th component represents the number of tokens in place p_i . $m(p)$ denotes the number of tokens in place p .

Graphically, places are represented by circles and transitions by bars. If $I(p, t) = 1$, we include a directed arc from place p to transition t , and $O(p, t) = 1$, a directed arc from transition t to place p . A marking assigns to each place a nonnegative integer. If a marking assigns to place p a nonnegative integer k , we say that p has k tokens, and we put k black dots (tokens) in place p .

The behavior of many systems can be described in terms of systems states and their changes. In order to simulate the dynamic behavior of a system, a state or marking in a Petri net is changed according to the following transition rules:

- (1) A transition is enabled if $m(p_i) \geq I(p_i, t_j)$ for any $p_i \in P$;
- (2) An enabled transition t can fire at marking m' , and its firing yields a new marking,

$$m(p) = m'(p) + O(p, t) - I(p, t), \text{ for arbitrary } p \text{ from } P.$$

The marking m is said to be reachable from m' . Given Z and its initial marking m_0 , the reachability set is the set of all marking reachable from m_0 through various sequences of transition firings and is denoted by $R(Z, m_0)$. Reachability set is a fundamental basis for studying the dynamic properties of a system.

A Petri net (Z, m_0) is said to be K -bounded or simply bounded if the number of tokens in each place does not exceed a finite number K for any marking reachable from m_0 . A Petri net (Z, m_0) is said to be safe if it is 1-bounded. For bounded Petri net, from the initial marking m_0 , we can obtain all possible reachable markings through firing all enabled transitions.

A Petri net (Z, m_0) is said to be live if, no matter what marking has been reached from m_0 , it is possible to ultimately fire any transition of the net by progressing through some further firing sequence. A live Petri net guarantees deadlock-free operation, no matter what firing sequence is chosen. The application to manufacturing is direct because, in general, the discrete event control should

not lead to deadlock or at least all possibilities for deadlock should be known. A Petri net (Z, m_0) is said to be reversible if, for each marking m in $R(Z, m_0)$, m_0 is reachable from m . Therefore, in a reversible net one can always get back to the initial marking.

III. BRANCH AND BOUND METHOD

Branch-and-bound methods have been developed in a variety of contexts, and under a variety of names, such as "backtrack programming" and "implicit enumeration". Essentially, the idea is to repeatedly break the set of feasible solutions into subsets, and to calculate bounds on the cost of the solutions contained within them. The bounds are used to discard entire subsets of solutions from further consideration. The simple but effective technique has scored a number of notable successes in practical computations.

Branch-and-bound algorithms are always 'primal' in the sense that they proceed from one feasible solution to another until optimality is verified. In fact they often find optimal or near optimal solutions early in the enumeration process and spend the majority of the time verifying optimality. Thus the user can be comforted by the expectation that termination before proof of optimality will likely yield a very good solution if not an optimal one [2].

In this paper, timed Petri nets are employed to model the aforementioned scheduling problem, and solve for an optimum schedule. The algorithm presented in this thesis combines the execution of the Petri net with a modified branch-and-bound scheme. Once the Petri net model of system is constructed, the evolution of the system can be described by changes in the marking of the Petri net. In other words, all possible behaviors of the system can be completely tracked by the reachability graph of the Petri net. Theoretically, therefore, an optimal schedule can be obtained by generating the reachability graph and finding the optimal path from the initial marking to the final marking. The path is a fired sequence of the transitions of the Petri net model.

Since the precedence relationships and resource constraints are incorporated in the Petri net model, only the feasible schedules would be obtained by executing the Petri net. Thus the search space is limited to that of feasible schedules. To further improve the search efficiency, an enabled transition would be fired only if it may lead to an optimum schedule. It should be pointed out that firing a transition during a search implies to explore a new schedule [15].

The branch-and-bound algorithm that finds an optimal or near optimal schedule is as follows. Given a Petri net model, it expands the reachability graph from the initial marking until the generated portion of the reachability graph touches the final marking. Once the final marking is reached, the path is constructed. Then the transition sequence of the path provides the order of starts of the activities.

The Method:

- Step 1: Find the initial marking (m_0) and final marking (m_f).
- Step 2: Set the upper bound (τ_{ub}) of makespan.
- Step 3: Let current marking $m_c = m_0$.
- Step 4: Find the enabled transitions of m_c . If enabled transition is in fired-sequence, then remove it from enabled transitions. If there is no enabled transition, then go to step 15.
- Step 5: Select one enabled transition to fire and put the others in waiting-list.
- Step 6: Record this fired transition to fired-sequence.
- Step 7: Generate the new marking (m_{new}) for this fired transition. Let $m_c = m_{new}$.
- Step 8: Calculate the actual cost (τ_c) from initial marking (m_0) to current marking (m_c), and estimate the cost from the current marking to the final marking and call it τ_e , the estimated makespan (τ_f) from the initial marking to the final marking which goes through the current marking is ($\tau_c + \tau_e$).
- Step 9: If $\tau_f > \tau_{ub}$, then go to Step 15.
- Step 10: If m_c has occurred before and if it may not make a shorter time ($\tau_c > \tau_b$), then go to Step 15.
- Step 11: If $m_c = m_f$, then go to Step 13.
- Step 12: Go to Step 4.
- Step 13: We find a feasible solution. Calculate the time makespan (τ_i).
- Step 14: If $\tau_i > \tau_{ub}$, then reject this solution. If $\tau_i < \tau_{ub}$, then this solution is selected. Let $\tau_{ub} = \tau_i$.
- Step 15: Go backward and get a transition from waiting-list. If there are no transitions in waiting-list, then terminate. Otherwise, fire this selected transition and go to Step 6.

In Step 2, use a large number as the upper bound to ensure that reasonably short makespan is not left out. In Step 4, each transition can be only fired once. In Step 5, in the case of the conflict jobs, we employ dispatching rules such as SPT (Shortest Processing Time) to select one transition to fire. In step 6, fired-sequence is a data structure which is used to record the solution that is being searched. In Step 8, the estimated makespan τ_f is a lower bound to all complete solutions descending from the current marking. The estimated value [15]

$\tau_e = \max_i \{ \text{The addition of operation time of those operations which are planned to be processed on the } i\text{th machine} \}, i = 1, 2, \dots, N. N \text{ is the total number of machines.}$

IV. A CASE STUDY

A job-shop system is a specific type of production system composed of a certain set of machines and a variety of jobs which must be produced using the machines. The manufacturing process of each job is specified as a sequence

of machines to visit, i.e., as a routing into the system. Any routing is allowable but is defined uniquely for each job. Further, the time spent by the jobs on the machines is assumed to be fixed and deterministic. Finally, the sequencing of the jobs on the machines is also assumed to be given, as well as the order with which jobs are loaded into the system (input sequencing).

Let us consider a job-shop system with 3 machines, denoted by M1, M2, M3 and 4 different types of jobs, denoted by Job 1, Job 2, Job 3, Job 4. Table 1 shows the job requirements of this example [1].

TABLE 1 JOB REQUIREMENTS OF THE EXAMPLE

Order	1st	2nd	3rd
Job 1	M1 (4U)	M2 (3U)	M3 (2U)
Job 2	M2 (1U)	M1 (4U)	M3 (4U)
Job 3	M3 (3U)	M2 (2U)	M1 (3U)
Job 4	M2 (3U)	M3 (3U)	M1 (1U)

As said in Section 2, we can obtain the Petri net model of the example (Table 1) as Figure 1 shows. Since the scheduling problem exists in the example, we may use the branch-and-bound algorithm to obtain the optimal schedule. Figure 2 shows the optimal result for the Example. In Figure 2, the manufacturing process of each job is supposed to be unique. That is to say, the executed sequences on each machine are fixed. Machine 1 processes Job 2 first, then Job 1 second, then Job 4 third, and Job 3 last. Machine 2 processes Job 2 first, then Job 4 second, then Job 3 third, and Job 1 last. Machine 3 processes Job 3 first, then Job 4 second, then Job 2 third, and Job 1 last.

This paper uses marked graphs to evaluate the steady-state cyclic performance. They are briefly discussed as follows. Marked graphs (also called decision-free Petri nets) are Petri nets such that each place has exactly one input and one output transition. In a marked graph, any two transitions do not share the same input place. Thus there is no conflict to simultaneously enable several transitions [9].

A marked graph has the following properties[12]:

- 1) The token count in a direct circuit is invariant under any firing.
- 2) It is live iff there is at least one token on each direct circuit.
- 3) A live marked graph is safe if each place belongs to a directed circuit whose token count is one.
- 4) There is a live and safe marking in any marked graph if it is strongly connected.
- 5) The maximal number of tokens that a place can have is equal to the minimal number of tokens placed by initial marking on directed circuits containing the place.

In a timed marked graph, each transition takes a real time to fire. When a transition is enabled, a firing is initiated by moving one token from each of the transition input place.

The token remains in the transition during the time of the firing execution and then the firing terminates by adding one token in each of the transition output place.

For timed marked graphs, there exists already the formula to find the system cycle time [7], [12], [13]. For a marked graph which has time delays in its transition and place [19], the system cycle time C is given by

$$C = \text{Max} \{ T_i / N_i : i = 1, 2, \dots, n \} \text{ where}$$

$$T_i = \text{Sum of the transition and place delays in circuit } \gamma_i,$$

$$N_i = \text{Total number of tokens in the places in circuit } \gamma_i,$$

$$n = \text{Number of circuits in the marked graph.}$$

Those circuits for which the cycle time is maximum are called bottleneck (critical) circuits. Indeed, those circuits are the ones that actually bound the throughput of the system. Two types of circuits in a marked graph are : 1) processing circuits — that model the manufacturing process of the sequence of each job. 2) command circuits — that model the sequencing of the jobs on the machines. If a circuit includes nodes of both processing and command circuits, then such circuit will be called a mixed circuit [9].

Since each place has exactly one input and output transition in a marked graph, we can easily find that the Petri net model of Figure 1 is not a marked graph because Places p_{25} , p_{26} , and p_{27} have more than one input and one output transition.

For job shop systems, we can directly derive the marked graph from the initial Petri net model after the schedule is determined. A general procedure can be stated as follows:

- (1) If a machine serves for i jobs, then change the number of places from 1 to i . Name these places as p^1_k , p^2_k , ..., p^i_k where k is the initial symbol for that machine.
- (2) According to the executed sequences on each machine, we draw an output arc from p^1_k to the transition which represents the first job processed by the machine, then draw an input arc to p^2_k from the transition which represents the first job finished by that machine. Based on the same method as above, we draw an output arc from p^2_k to the transition which represents the second job processed by the machine, then draw an input arc to p^3_k from the transition which represents the second job finished by that machine. Repeat the above procedure until we draw an output arc from p^i_k to the last job processed by that machine. Then we draw a last arc from the last job finished by that machine to p^1_k .
- (3) Put a token in p^1_k .

We can apply this general procedure to our example. Since Machine 1 processes 4 jobs, we draw 4 places (p^1_{25} , p^2_{25} , p^3_{25} , and p^4_{25} where "25" represents Machine 1). Since it processes Job 2 first, we draw an output arc from p^1_{25} to t_9 and draw an input arc to p^2_{25} from t_{10} . Since Machine 1 processes Job 1 second, we draw an output arc from p^2_{25} to t_1 and draw an input arc to p^3_{25} from t_2 . Because it processes

Job 4 third, we draw an output arc from p^3_{25} to t_{23} and draw an input arc to p^4_{25} from t_{24} . Since it processes Job 3 last, we draw an output arc from p^4_{25} to t_{17} and draw an input arc to p^1_{25} from t_{18} . We do the same procedures for Machines 2 and 3. Finally, we put three tokens in places p^1_{25} , p^1_{26} , and p^1_{27} . According to the above procedure, we can obtain the marked graph of the FMS using the optimal scheduling of the example, as shown in Figure 3.

In Figure 3, each place has exactly one input and one output transition. Therefore, the net is a marked graph. The firing times are shown in Table 2.

TABLE 2 OPERATION TIMES OF FIGURE 3

Operatio n	time	Operatio n	time	Operatio n	time
t_1	0	t_9	0	t_{17}	0
t_2	4	t_{10}	4	t_{18}	1
t_3	0	t_{11}	0	t_{19}	0
t_4	3	t_{12}	4	t_{20}	3
t_5	0	t_{13}	0	t_{21}	0
t_6	2	t_{14}	3	t_{22}	3
t_7	0	t_{15}	0	t_{23}	0
t_8	1	t_{16}	2	t_{24}	1

There are totally 49 elementary circuits (including four processing circuits and three command circuits). The $\max\{T_i/N_i\}$ is equal to 12 Units. Therefore, the maximum cycle time is 12 Units.

According to Figure 2, we can find that makespan of the optimal scheduling of the example is 14 Units. Based on Figure 3, the cycle time of the system is 12 Units. It is reasonable, because it is a long-term run performance. In Figure 2, Machine 3 finished Job 1 (i.e., the first part of each job) at the end of 14-th Unit. But Machine 2 is idle in 12-th Unit, and Machine 2 can process the first operation of the second part of the Job 2 (i.e., the second part of each job). Therefore, the cycle time (12 Units) < the makespan (14 Units) is reasonable. It is clear that if the cycle time is greater than the makespan, then the cycle time is incorrect.

V. CONCLUSION

This paper discusses the scheduling problem of an FMS. We construct the complete Petri net model of an FMS. A firing sequence of the Petri net from the initial marking to the final one can be seen as a schedule of the modeled FMS. In order to obtain an optimal schedule and avoid wasting time, we use the branch-and-bound method. This method can give not only an optimal solution, but also can reduce the complexity because only the feasible solutions can be obtained.

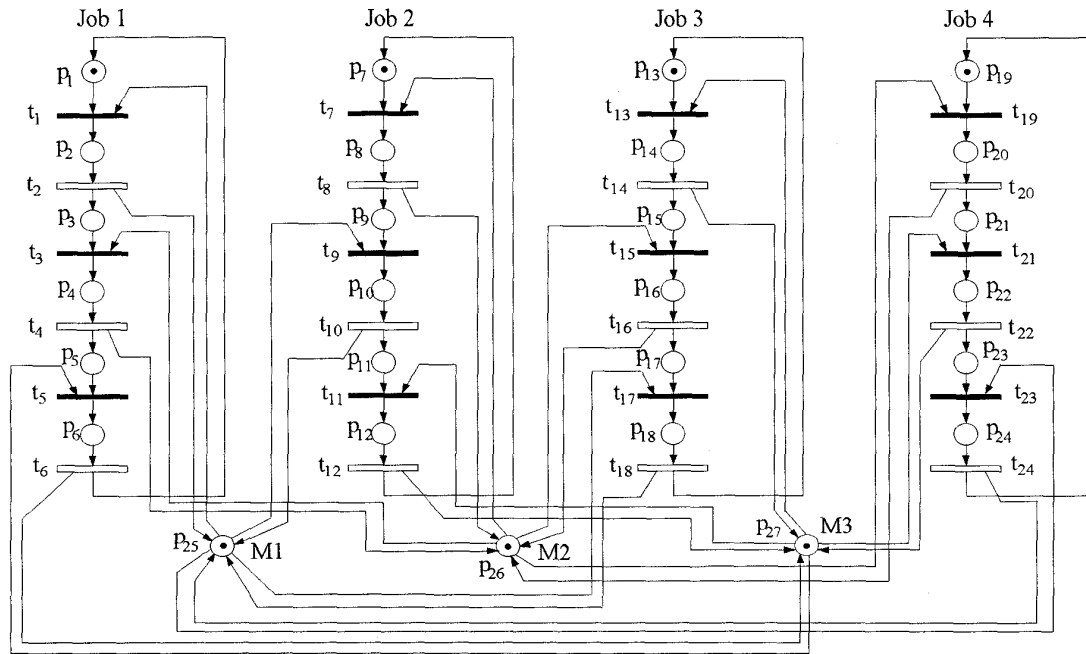


Figure 1 PN Model of the Example

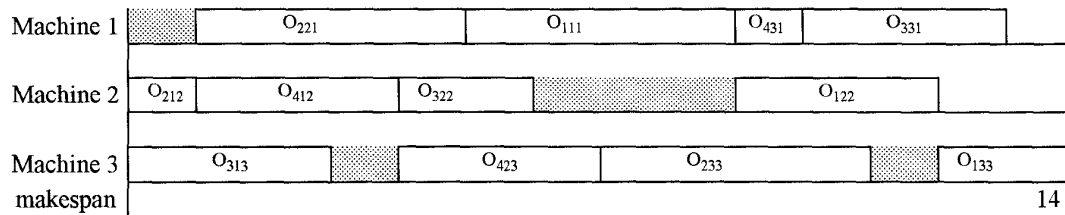


Figure 2 The optimal schedule of the Example

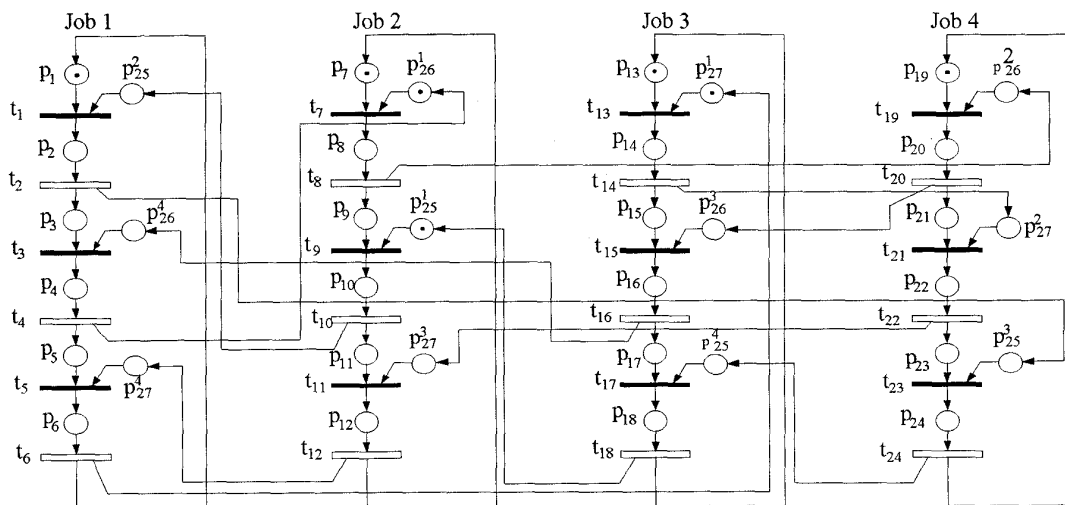


Figure 3 A marked graph of the optimal scheduling of the Example

The branch-and-bound method is used to find an optimal schedule. Given a Petri net model, it expands the reachability graph from initial marking until the generated portion of the reachability graph contains the final marking. Once the final marking is reached, the path is constructed. Then the transition sequence of the path provides the order of starts of the activities.

In order to get the long-term production rate of a system given the schedule, marked graphs are constructed. They are a special class of timed Petri nets used for modeling and analyzing choice-free concurrent job-shop systems. The modeling allows for evaluating the long-term periodic performance of the system under a deterministic and cyclic production process. Given any fixed processing times, the productivity (i.e., production rate) of the system can be determined from the initial state.

There are two areas which need to be explored in the future study:

- 1) The failure rates of machines and AGVs are not considered in this paper. Therefore, an FMS that includes the failure rates of machines and AGVs requires future research.
- 2) To find a method which can give the cycle time of a system easily, e.g., using linear programming method as shown in [9].

VI. REFERENCES

- [1] Baker, K. R., *Introduction to Sequencing and Scheduling*. New York: Wiley, 1974.
- [2] Christofides, N., Mingozzi, A., Toth, P. and Sandi, C., *Combinatorial Optimization*. New York: John Wiley & Sons, 1978.
- [3] Chu, C. and Proth, J. M., "A Splitting-up Approach to simply Job-Shop Scheduling Problems", *Int. Journal of Production Research*, 30(4), pp. 859-870, 1992.
- [4] Conway, R., Maxwell, W. L. and Miller, L. W., *Theory of Scheduling*. Ontario: Addison-Wesley, 1967.
- [5] Cooper, L. and Steinberg, D., *Introduction to Methods of Optimization*. Philadelphia: Saunders, 1970.
- [6] DiCesare, F. and Desrochers, A., "Modeling, Control, and Performance Analysis of Automated Manufacturing Systems using Petri Nets", *Control and Dynamic Systems*, pp. 121-172, 1991.
- [7] Freedman, P., "Time, Petri nets, and robotics", *IEEE Trans. Robotics Automation*, 7(4), pp. 417-433, 1991.
- [8] Gupta, D. and Buzacott, J. A., "A Framework for Understanding Flexibility of Manufacturing Systems", *Journal of Manufacturing Systems*, 8(2), pp. 89-95, 1989.
- [9] Hillion, H. P., and Proth, J. M., "Performance Evaluation of Job-Shop Systems Using Timed Event-Graphs", *IEEE Trans. on Automatic Control*, 34(1), pp. 149-154, 1989.
- [10] Lee, D. Y., *Scheduling and Supervisory Control of Flexible Manufacturing Systems Using Petri Nets and Heuristic Search*, Ph.D. Dissertation, Rensselaer Polytechnic Institute, Troy, NY, May 1993.
- [11] Lee, D. Y. and DiCesare, F., "Petri Net-Based Heuristic Scheduling for Flexible Manufacturing Systems", in *Petri Nets in Flexible & Agile Automation*, M. C. Zhou (Ed.), pp. 149-187, Boston, MA: Klumer Academic, 1995.
- [12] Murata, T., "Petri Nets : Properties, Analysis and Applications", *Processings of the IEEE*, 77(4), 1989.
- [13] Ramchandani, C., *Analysis of asynchronous concurrent systems by timed Petri nets*, Ph.D. dissertation, MIT, Cambridge, MA, May 1974.
- [14] Rodammer, F. A. and White, K. P. Jr., "A Recent Survey of Production Scheduling", *IEEE Trans. on Systems, Man, and Cybernetics*, 18(6), pp. 841-851, 1988.
- [15] Shen, L., Chen, Q., Luh, J. Y., Chen, S. C. and Zhang, Z., "Truncation of Petri Net Models of Scheduling Problems for Optimun Solutions", *Japan/USA Symposium on Flexible Automation*, pp. 1681-1688, 1992.
- [16] Xiong, H. H., Zhou, M. C. and Manikopoulos, C. N., "Modeling and Performance Analysis of Medical Services Systems Using Petri Nets", *Proc. of 1994 IEEE Conf. on Systems, Man and Cybernetics*, San Diego, TX, pp. 2339-2342, Oct. 1994.
- [17] Xiong, H. H., Zhou, M. C. and Manikopoulos, C. N., "Scheduling Flexible Manufacturing Systems Based on Timed Petri Net and Fuzzy Dispatching Rules", *INRIA/IEEE Conf. on Emerging Technologies and Factory Automation*, Paris, Oct. 1995.
- [18] Zhou, M. C. (Ed.), *Petri Nets in Flexible and Agile Autonation*, Boston, MA: Klumer Academic, 1995.
- [19] Zhou, M. C., McDermott, K. and Patel, P. A., "Petri Net Synthesis and Analysis of a Flexible Manufacturing System Cell", *IEEE Trans. on Systems, Man, and Cybernetics*, 23(2), pp. 523-531, 1993.